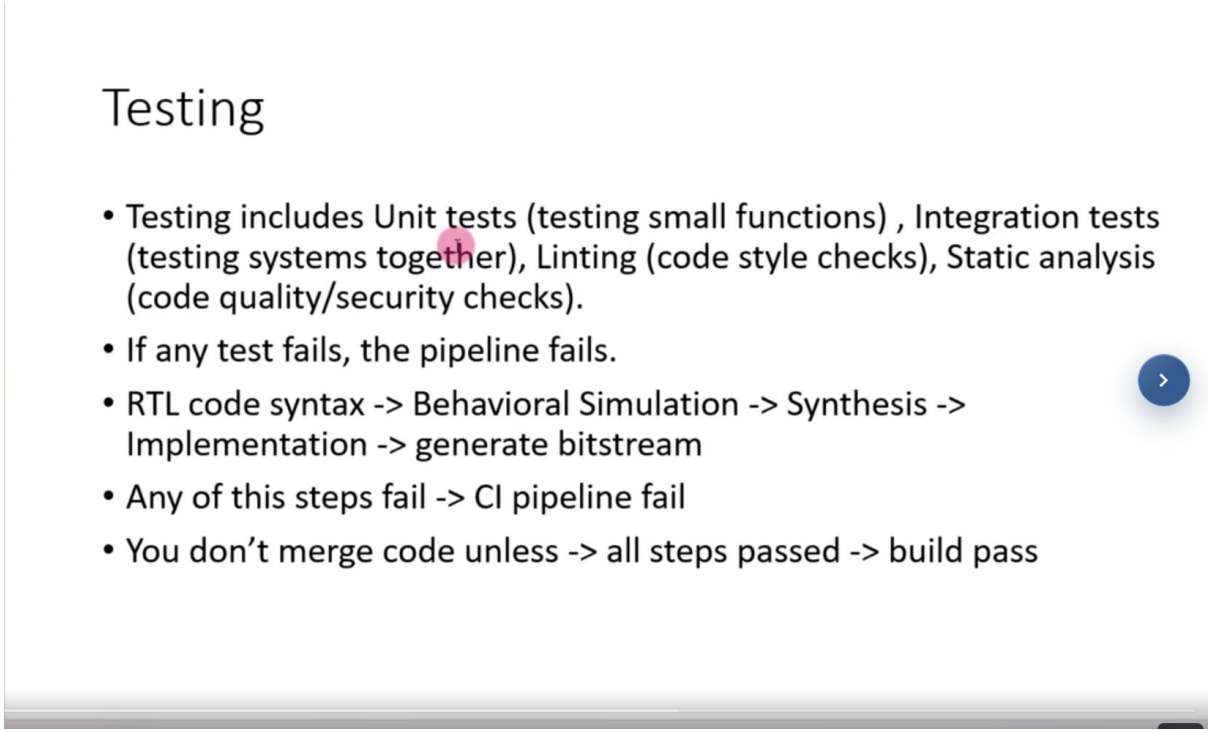
Infrastructure Setup : Semiconductor

Course Link : https://namaste-fpga.com/student/learn/105

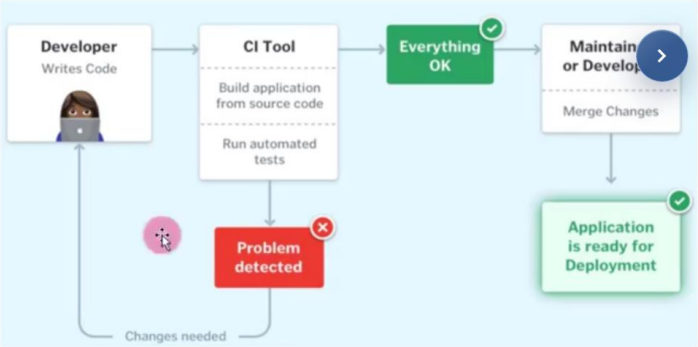
My GitHub Infrastructure Setup Workflow and all stuff : https://github.com/ssbagi/Reusable-Chip-Platform

So, there is no CCI Violation at all. All the methods and understanding is Publicly available.

1. Git Version Control
2. Jenkins CI/CD



1. Developer writes code.
2. Developer pushes code to a branch.
3. CI system triggers automatically.
4. CI builds the project.
5. CI runs tests.
6. If everything passes → merge allowed.
7. If something fails → fix before merging.



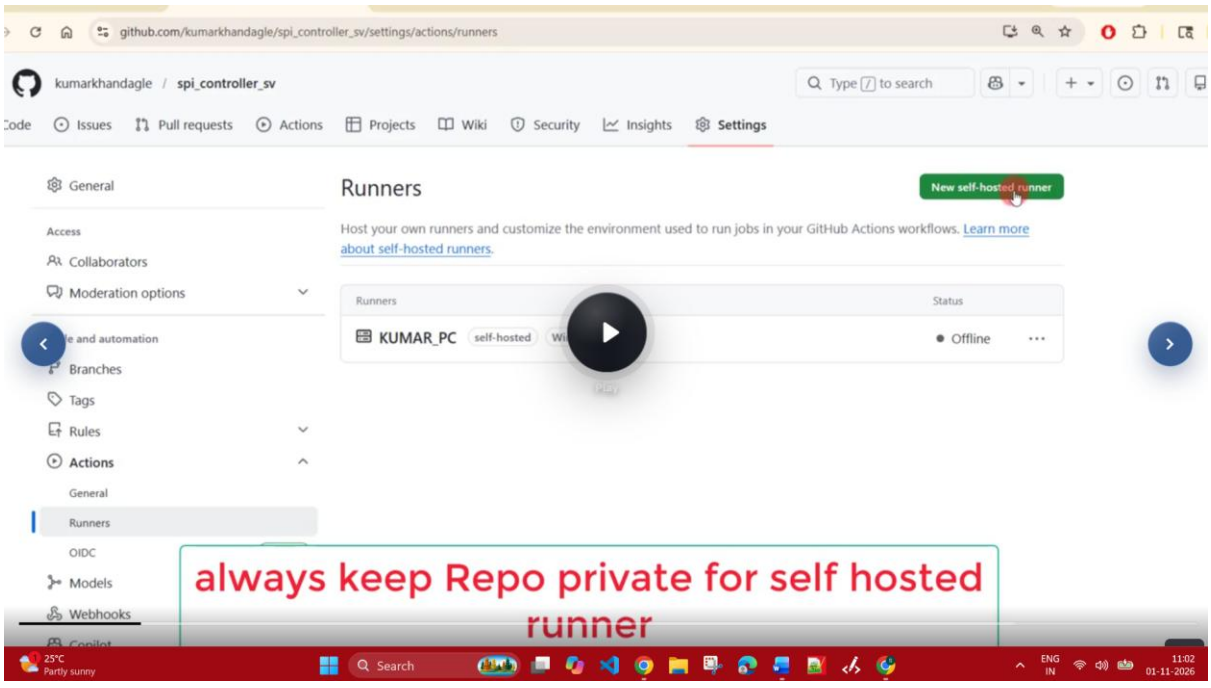
CD

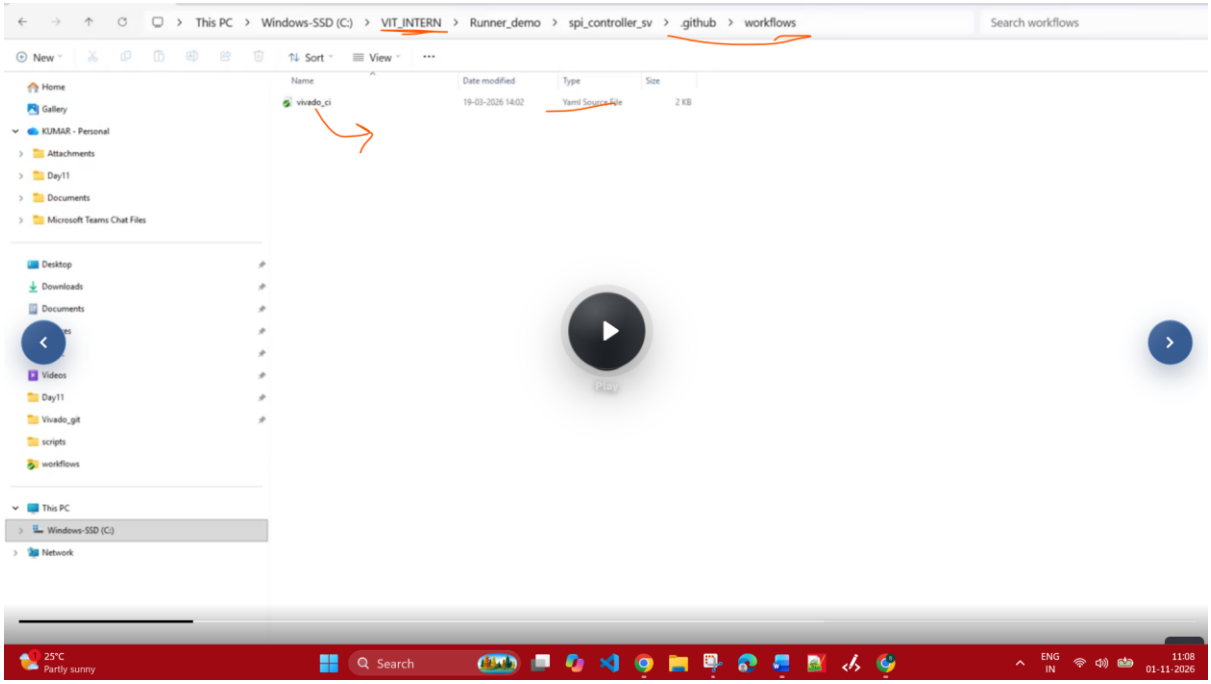
- After CI completes, we have Bitstream file, Timing reports, Utilization reports, Simulation logs
- If you have remote lab infrastructure CD can upload the bitstream to a programming server, Flash the FPGA board automatically, Run hardware validation tests, Capture hardware logs
- Create a Git tag (e.g., v1.2.0) -> Ensure the tag points to the exact commit -> Lock that version permanently -> commit becomes an official release reference.

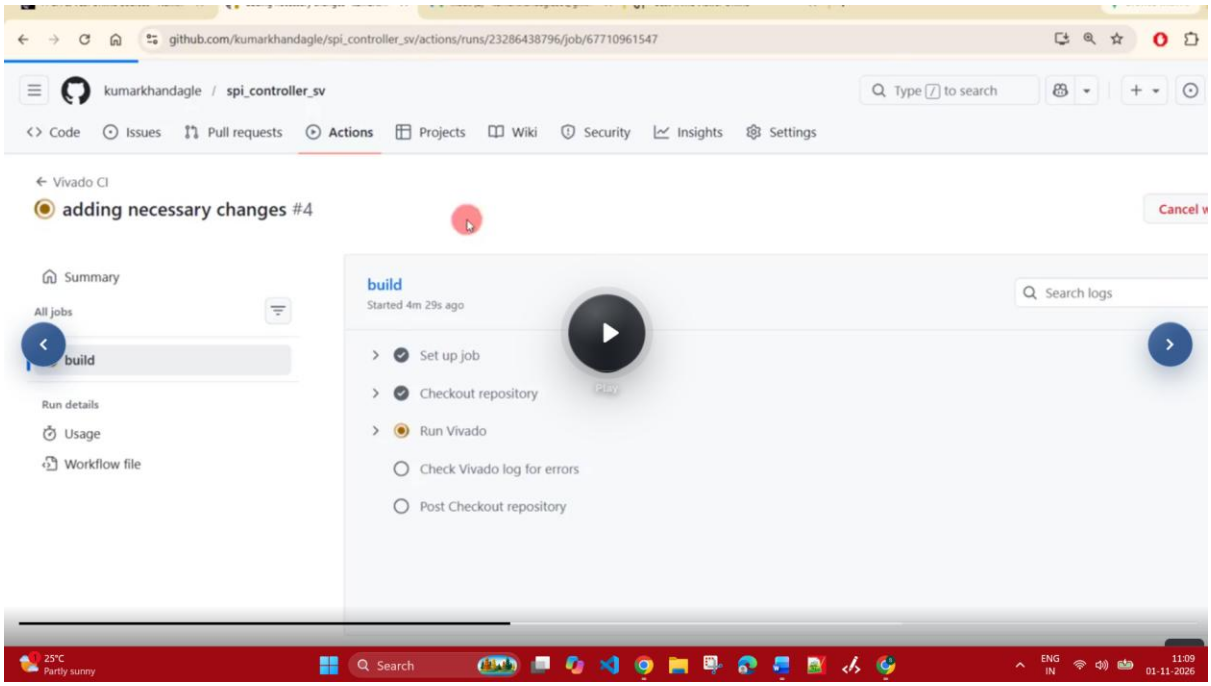
CI tools

- GitHub Actions
- GitLab CI
- CircleCI
- Jenkins
- Azure DevOps

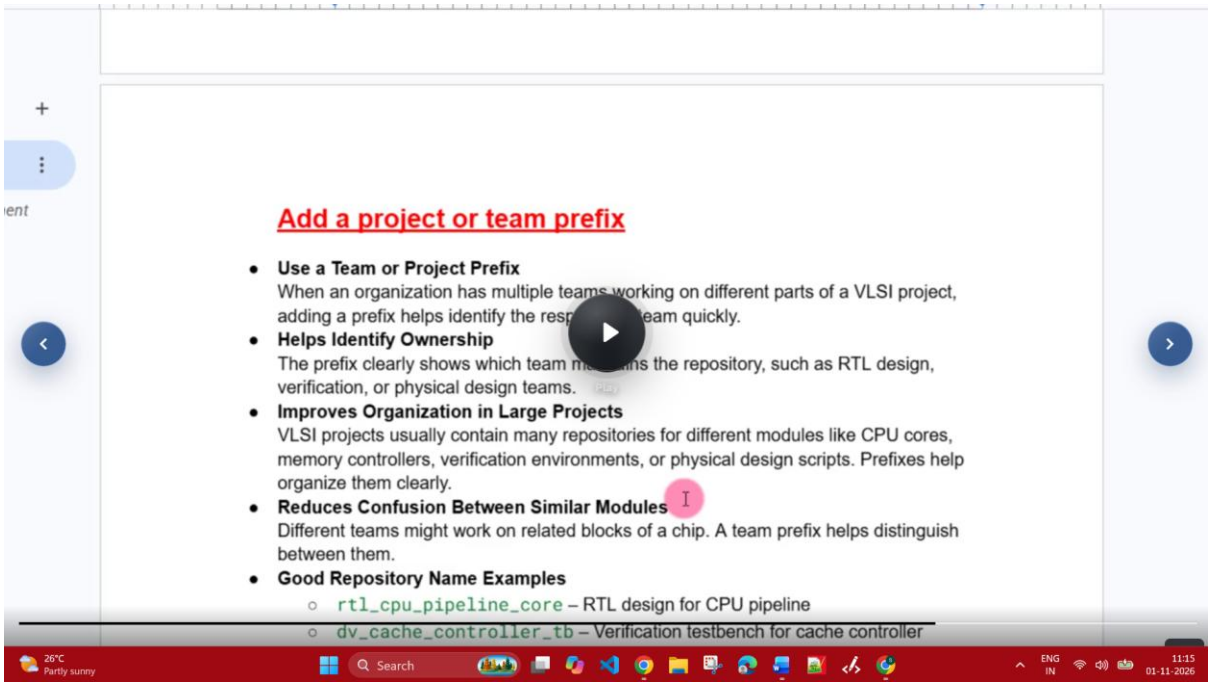
GitHub Flow Actions :




This PC > Windows-SSD (C:) > VIT_INTERN > Runner_demo > spi_controller_sv > .github > workflows
Search workflows
New
Sort
View
Name
Date modified
Type
Size
vivado_ci
19-03-2026 14:02
Yaml Source File
2 KB
Home
Gallery
KUMAR - Personal
Attachments
Day11
Documents
Microsoft Teams Chat Files
Desktop
Downloads
Documents
Videos
Day11
Vivado_git
scripts
workflows
This PC
Windows-SSD (C:)
Network
Play
25°C
Partly sunny
Search
ENG
IN
11:08
01-11-2026

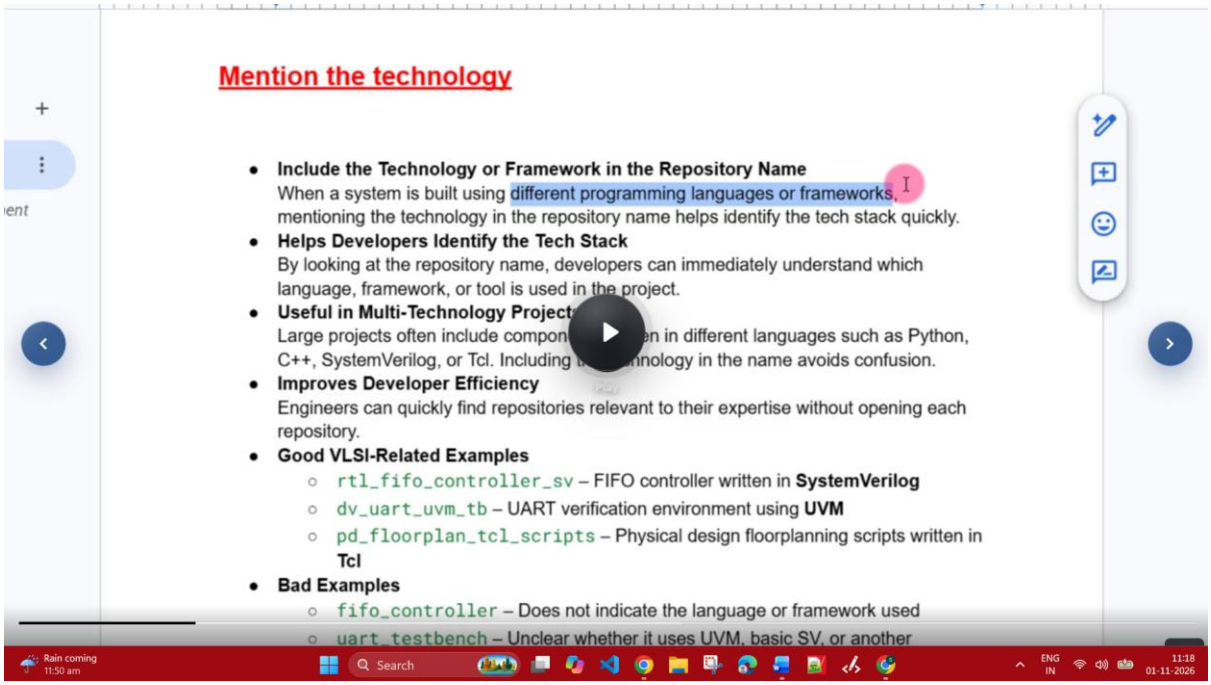

github.com/kumarkhandagle/spi_controller_sv/actions/runs/23286438796/job/67710961547
kumarkhandagle / spi_controller_sv
Type / to search
Code
Issues
Pull requests
Actions
Projects
Wiki
Security
Insights
Settings
Vivado CI
adding necessary changes #4
Cancel w
Summary
All jobs
build
Run details
Usage
Workflow file
build
Started 4m 29s ago
Search logs
Set up job
Checkout repository
Run Vivado
Check Vivado log for errors
Post Checkout repository
Play
25°C
Partly sunny
Search
ENG
IN
11:09
01-11-2026

See it is already there : The way I showed the workflow concept stuff DL1, DL2 and DL3 or Leaf to Submodule to Block to SoC level and stuff.

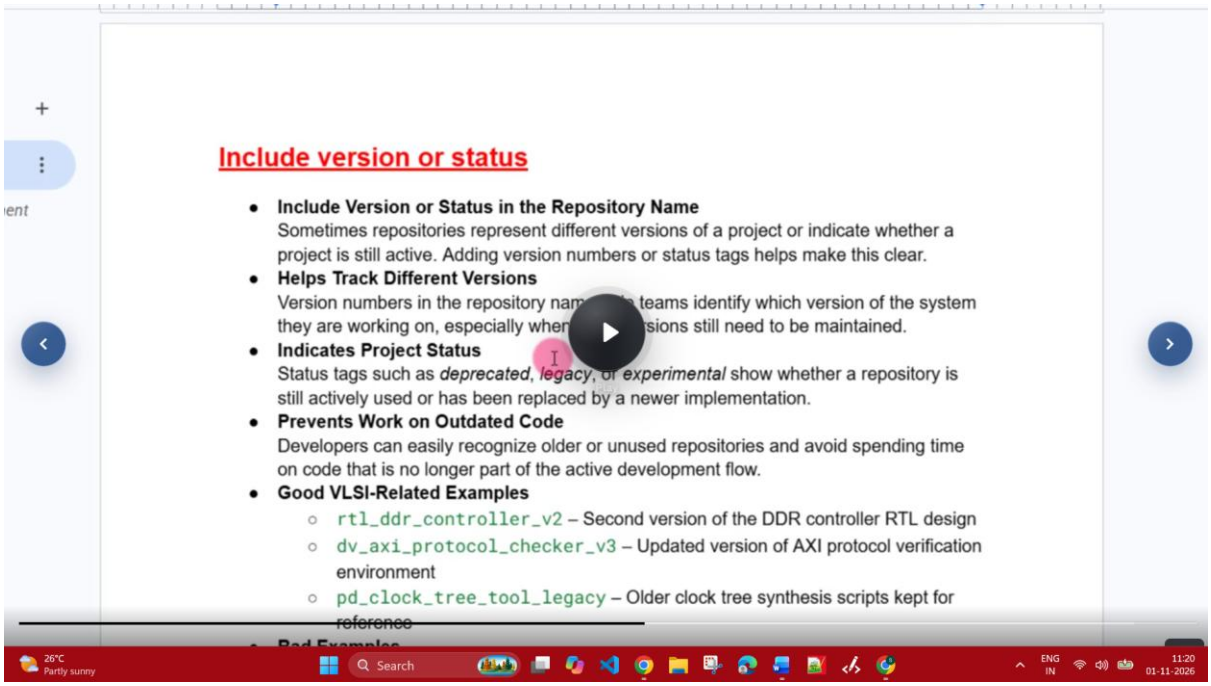


- **Reduces Confusion Between Similar Modules**
  Different teams might work on related blocks of a chip. A team prefix helps distinguish between them.
- **Good Repository Name Examples**
  - `rtl_cpu_pipeline_core` – RTL design for CPU pipeline
  - `dv_cache_controller_tb` – Verification testbench for cache controller
  - `pd_powergrid_generator` – Power grid generation scripts for physical design
- **Bad Repository Name Examples**
  - `project1` – Too vague, no information about the content
  - `cpu_stuff` – Unclear and informal naming
  - `final_repo_new` – Does not explain the purpose or ownership
- **Benefit**
  Engineers can quickly identify the team and purpose of the repository, making it easier to search, manage, and collaborate in large chip design projects

See even when I wrote the post on Friday, I mean the different frameworks learning and languages all are told or I mean existing ones only. Whatever I have said is entirely True and correct only.

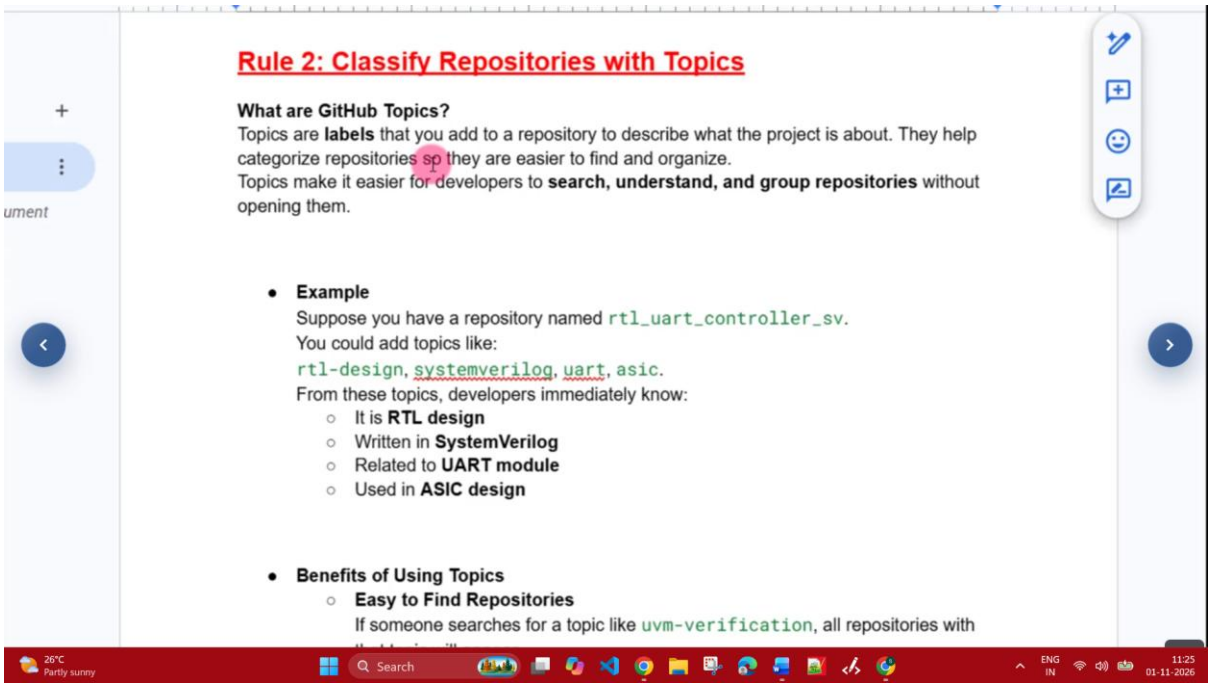


Version Control :



All these names follow the rules because they:

- include a **team prefix**
- are **descriptive**
- sometimes include **technology/framework**
- may include **version**
- use **only letters, numbers, and underscores**
- clearly indicate the **type of project/module**.



Rule 2: Classify Repositories with Topics

What are GitHub Topics?
Topics are **labels** that you add to a repository to describe what the project is about. They help categorize repositories so they are easier to find and organize.
Topics make it easier for developers to **search, understand, and group repositories** without opening them.

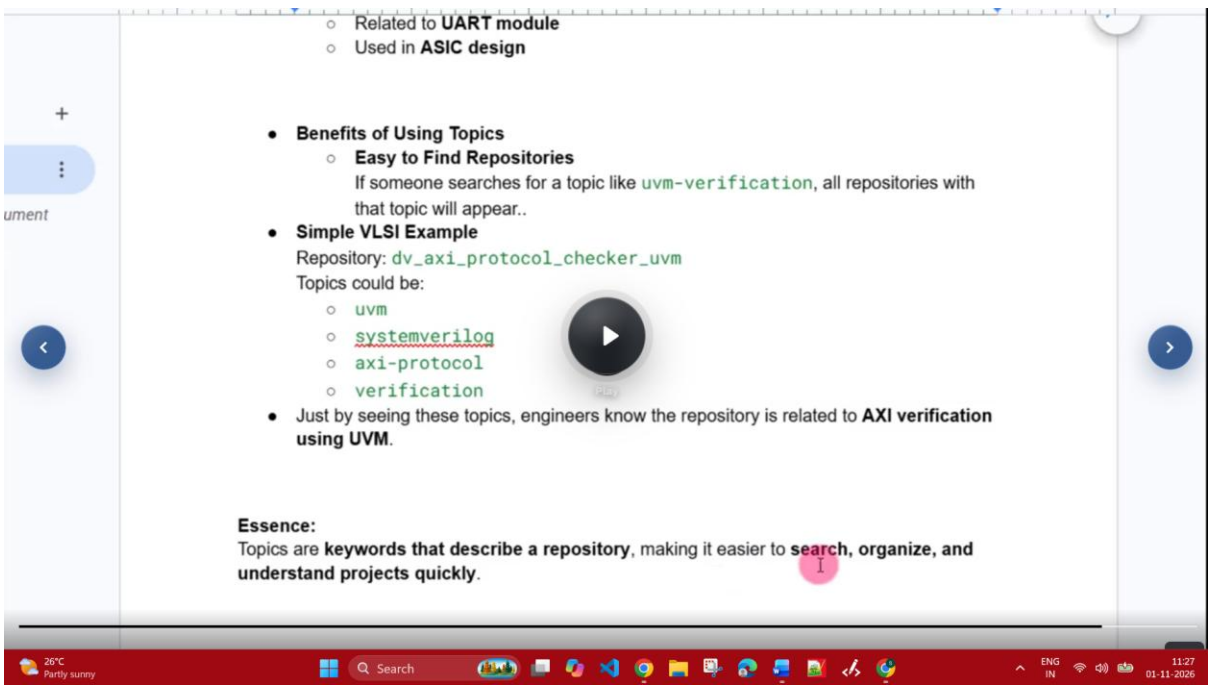
- **Example**
  Suppose you have a repository named `rtl_uart_controller_sv`.
  You could add topics like:
  `rtl-design`, `systemverilog`, `uart`, `asic`.
  From these topics, developers immediately know:
  - It is **RTL design**
  - Written in **SystemVerilog**
  - Related to **UART module**
  - Used in **ASIC design**

- **Benefits of Using Topics**
  - **Easy to Find Repositories**
    If someone searches for a topic like `uvm-verification`, all repositories with

- Related to **UART module**
- Used in **ASIC design**

- **Benefits of Using Topics**
  - **Easy to Find Repositories**
    If someone searches for a topic like `uvm-verification`, all repositories with that topic will appear..
- **Simple VLSI Example**
  Repository: `dv_axi_protocol_checker_uvm`
  Topics could be:
  - `uvm`
  - `systemverilog`
  - `axi-protocol`
  - `verification`
- Just by seeing these topics, engineers know the repository is related to **AXI verification using UVM**.

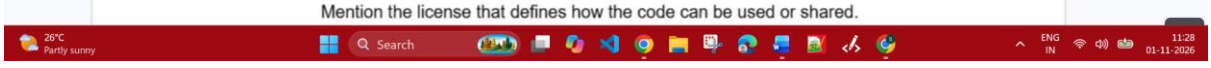
Essence:
Topics are **keywords that describe a repository**, making it easier to **search, organize, and understand projects quickly**.



Use README.md to Document the Repository

- **What is README.md?**
  `README.md` is a file placed in the root of a GitHub repository that explains the project. It is the **first thing people see** when they open the repository.
  It gives a **quick overview of the project**, what it does, and how someone can start using or contributing to it.

- **Important Information to Include**
  - **Project Description**
    A short explanation of what the project is about and what problem it solves.
    Example: An RTL implementation of an SPI controller for ASIC design.
  - **Setup Instructions**
    Steps required to set up the environment or tools needed to run or use the project.
  - **Usage Examples**
    Show how the project can be used. This helps users quickly understand how to run simulations, scripts, or tools.
  - **Contribution Guidelines**
    Explain how others can contribute to the project, such as adding features, fixing bugs, or improving documentation.
  - **License Information**
    Mention the license that defines how the code can be used or shared.



See even the directory structure also in the same way :

